

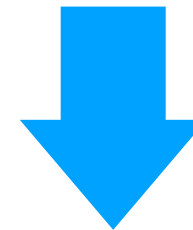
Machine Learning: Regression

Tien-Lam Pham

lam.phamtien@phenikaa-uni.edu.vn

Intelligent Machine

INPUT
Information
(X)



Programming

Give machine set of rules

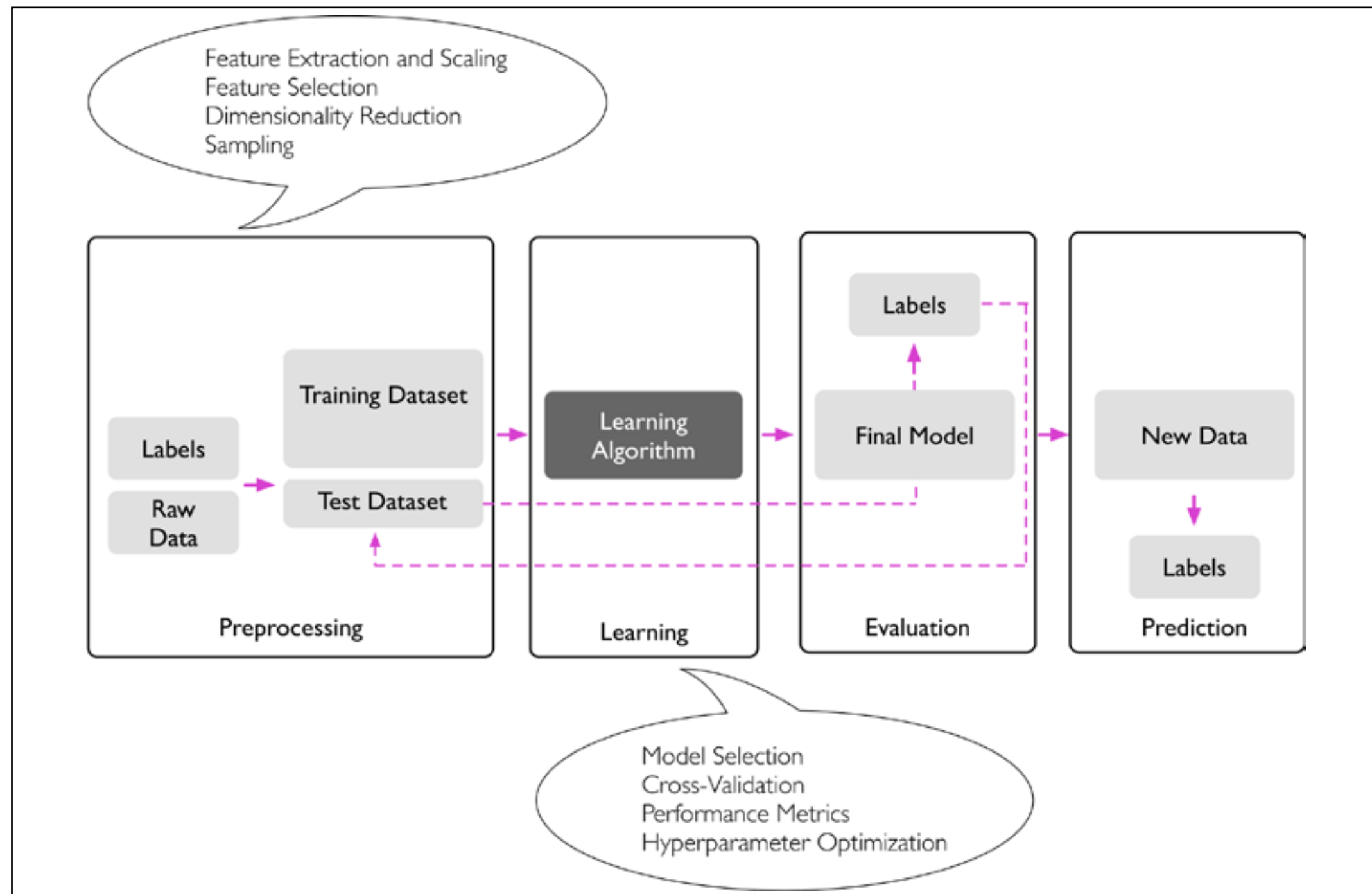
Learning from data
(Machine learning)

Learning from data: PC tự động
khái hoá các tri thức ẩn từ dữ
liệu để thực thi một chức năng
“trí tuệ” nào đó (phán đoán ->
hành động)

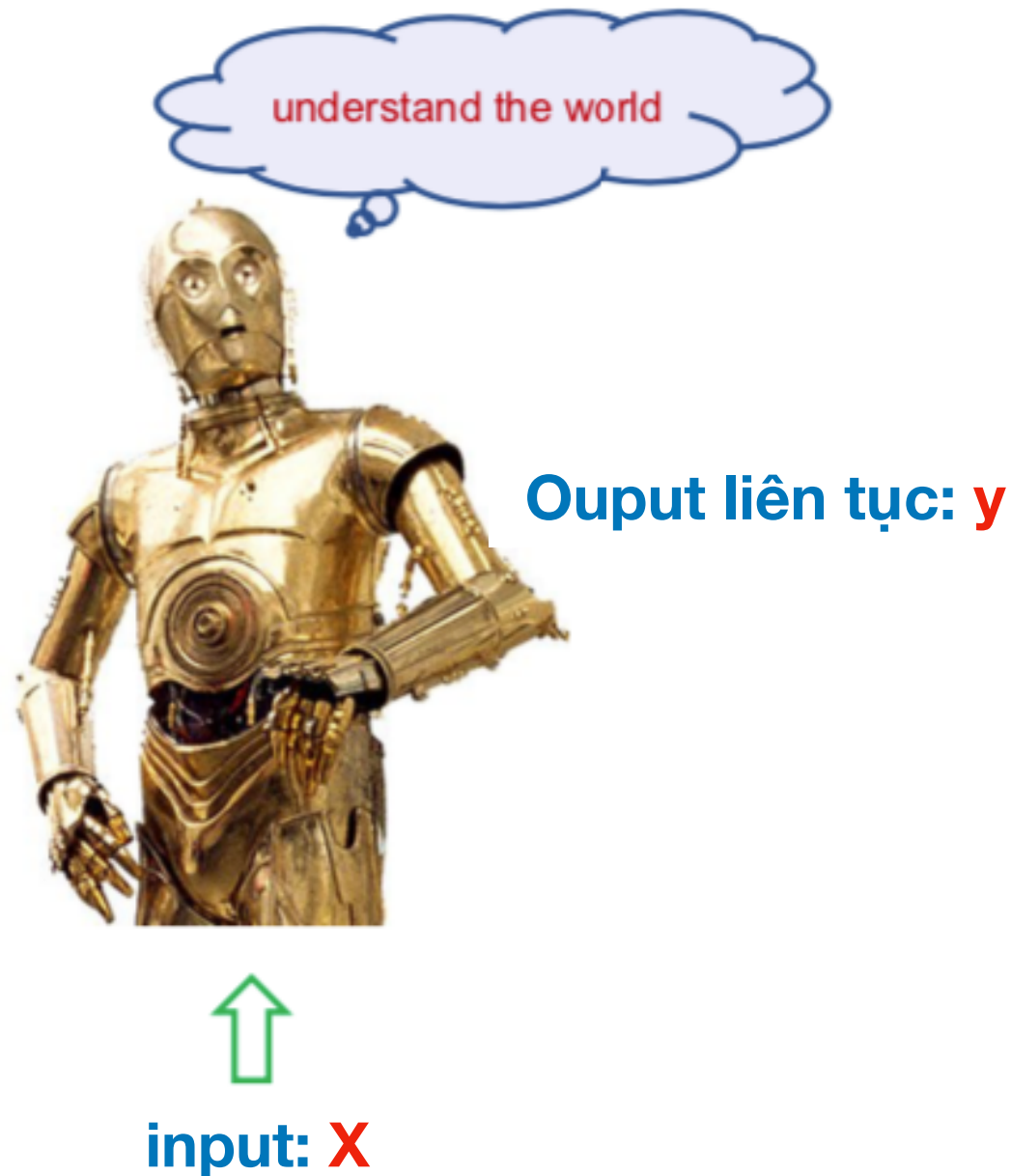


OUTPUT
Prediction (y)
Action (0, 1)

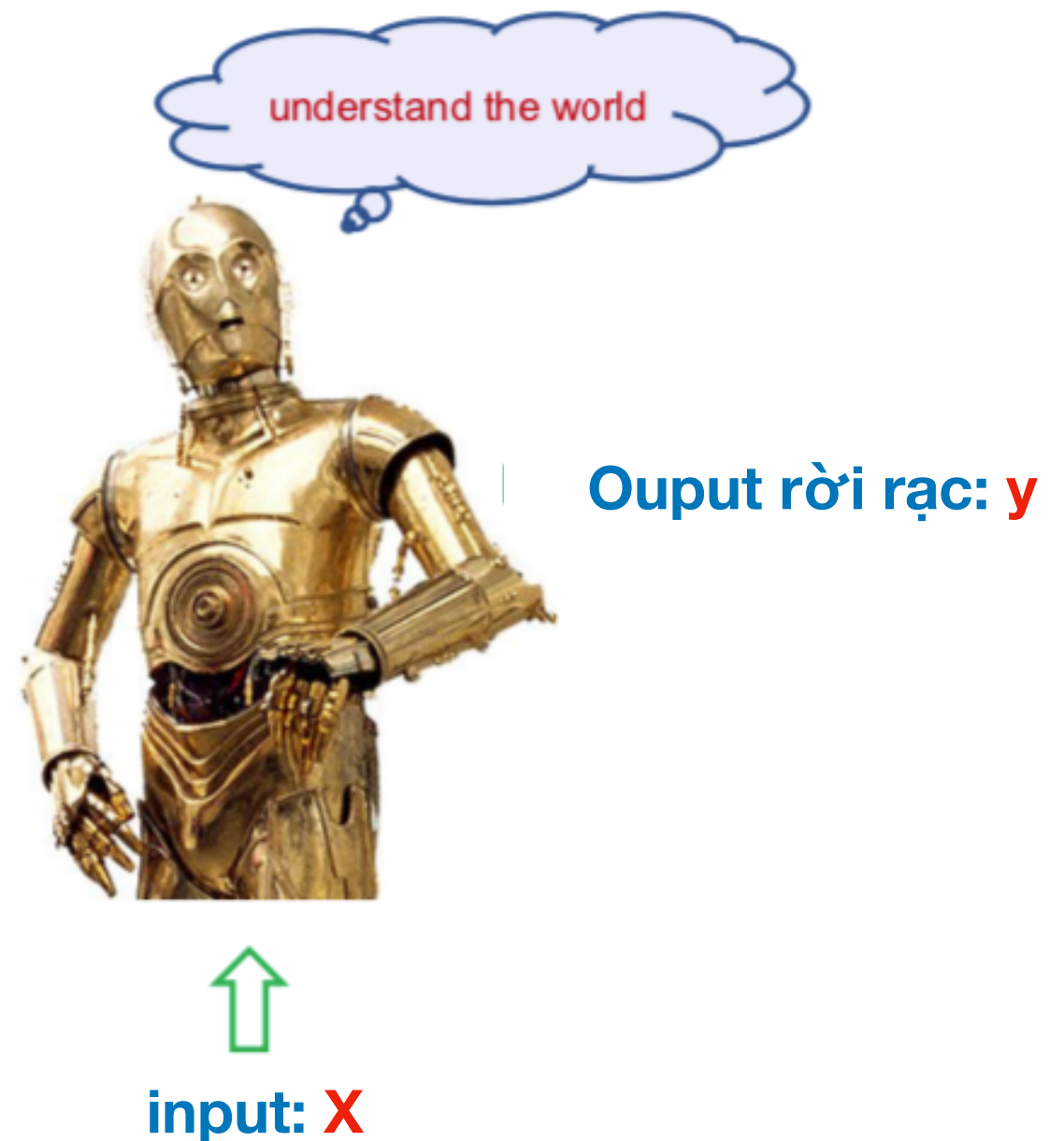
Roadmap for Building Machine Learning Systems



Classification vs. Regression



**Bài toán hồi qui
(regression)**



**Bài toán phân loại
(classification)**

Data-Driven Approach

(1) Problem setting

(2) Data collection

$$D = \{(x_i, y_i), i = 1, 2, \dots, m\}$$

(3) Modeling and Training Models

```
def train(images, labels):  
    # Machine learning!  
    return model
```

(4) Model selection

(5) Deploy suitable model (Using the best model to make prediction)

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

Machine Learning

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ."

Tom Mitchell

“Một chương trình máy tính được cho là có khả năng học từ kinh nghiệm (dữ liệu), E , để thực hiện một chức năng T với hiệu năng P , nếu nó thực hiện chức năng T tốt hơn (được đo bằng hiệu năng P) nếu có mặt dữ liệu E ”

Example: Linear Regression

T: predict profit of startups

E: data

P: Accuracy

	A	B	C	D	E	
1	R&D Spend	Administration	Marketing Spend	State	Profit	
2	165349.2	136897.8	471784.1	New York	192261.83	
3	162597.7	151377.59	443898.53	California	191792.06	
4	153441.51	101145.55	407934.54	Florida	191050.39	
5	144372.41	118671.85	383199.62	New York	182901.99	
6	142107.34	91391.77	366168.42	Florida	166187.94	
7	131876.9	99814.71	362861.36	New York	156991.12	

$$D = \{(x_i, y_i), i = 1, 2, \dots, m\}$$

$$x_i = (x_{i1}, x_{i2}, x_{i3})$$

y_i = profit of startup i

x_{i1} : R&D spend of startup i

x_{i2} : Administration fee of startup i

x_{i3} : Marketing fee of startup i

Example: Linear Regression

Step 1: Data collection

$$D = \{(x_i, y_i), i = 1, 2, \dots, m\}$$

$$x_i = (x_{i1}, x_{i2}, x_{i3})$$

$$y_i = \text{profit of startup } i$$

Step 2: Modeling and Training model

$$\hat{y}_i = f(x_{i1}, x_{i2}, x_{i3})$$

$$= w_1 \times x_{i1} + w_2 \times x_{i2} + w_3 \times x_{i3}$$

$$= \vec{x}_i \cdot \vec{w}$$

Example: Linear Regression

Step 2: Modeling and Training model

$$\begin{aligned}\hat{y}_i &= f(x_{i1}, x_{i2}, x_{i3}) \\ &= w_1 \times x_{i1} + w_2 \times x_{i2} + w_3 \times x_{i3} \\ &= \vec{x}_i \cdot \vec{w}\end{aligned}$$

Loss Function (Cost function)

$$\begin{aligned}L(w_1, w_2, w_3) &= \frac{1}{2m} \sum_i^m (\hat{y}_i - y_i)^2 \\ &= \frac{1}{2m} \sum_i^m (w_1 x_{i1} + w_2 x_{i2} + w_3 x_{i3} - y_i)^2\end{aligned}$$

Training = finding w which minimizes L

Example: Vector Representation

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & x_{m3} \end{bmatrix}$$

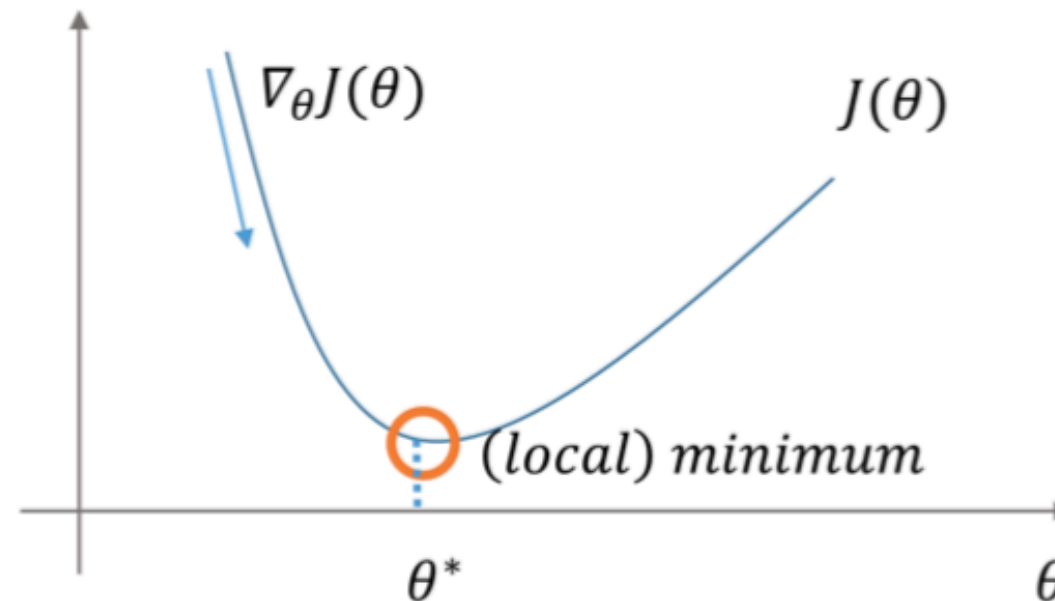
$$\hat{Y} = X \cdot w^T$$

$$= \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & x_{m3} \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix}$$

$$L = \frac{1}{2m} (\hat{Y} - Y)^T \cdot (\hat{Y} - Y)$$

Gradient descent

- Gradient descent is a way to minimize an objective function $J(\theta)$
 - $\theta \in \mathbb{R}^d$: model parameters
 - η : learning rate
 - $\nabla_{\theta} J(\theta)$: gradient of the objective function with regard to the parameters
- Updates parameters **in opposite direction** of gradient.
- Update equation: $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta)$



Gradient Descent

$$w_1 := w_1 - \eta \frac{\partial L}{\partial w_1}$$

$$w_2 := w_2 - \eta \frac{\partial L}{\partial w_2}$$

$$w_3 := w_3 - \eta \frac{\partial L}{\partial w_3}$$

$$\begin{aligned} \frac{\partial L}{\partial w_1} &= \frac{1}{m} \sum_i^m x_{i1} (\hat{y} - y_i) \\ &= \frac{1}{m} x_1^{(c)T} \cdot ((\hat{Y}) - Y) \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial w_2} &= \frac{1}{m} \sum_i^m x_{i2} (\hat{y} - y_i) \\ &= \frac{1}{m} x_2^{(c)T} \cdot ((\hat{Y}) - Y) \end{aligned}$$

$$\begin{aligned} \frac{\partial L}{\partial w_3} &= \frac{1}{m} \sum_i^m x_{i3} (\hat{y} - y_i) \\ &= \frac{1}{m} x_3^{(c)T} \cdot ((\hat{Y}) - Y) \end{aligned}$$

Gradient Descent: Vectorization

$$w_1 := w_1 - \eta \frac{\partial L}{\partial w_1}$$

$$w_2 := w_2 - \eta \frac{\partial L}{\partial w_2}$$

$$w_3 := w_3 - \eta \frac{\partial L}{\partial w_3}$$

$$W^T := W^T - \eta \nabla_W L$$

$$\nabla_W L = X^T \cdot (\hat{Y} - Y)$$

Gradient descent: full batch

- Computes gradient with the **entire** dataset.
- Update equation: $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta)$

```
for i in range(nb_epochs):  
    params_grad = evaluate_gradient(  
        loss_function, data, params)  
    params = params - learning_rate * params_grad
```

- Pros:
 - Guaranteed to converge to **global** minimum for **convex** error surfaces and to a **local** minimum for **non-convex** surfaces.
- Cons:
 - **Very slow.**
 - Intractable for datasets that **do not fit in memory.**
 - **No online learning.**

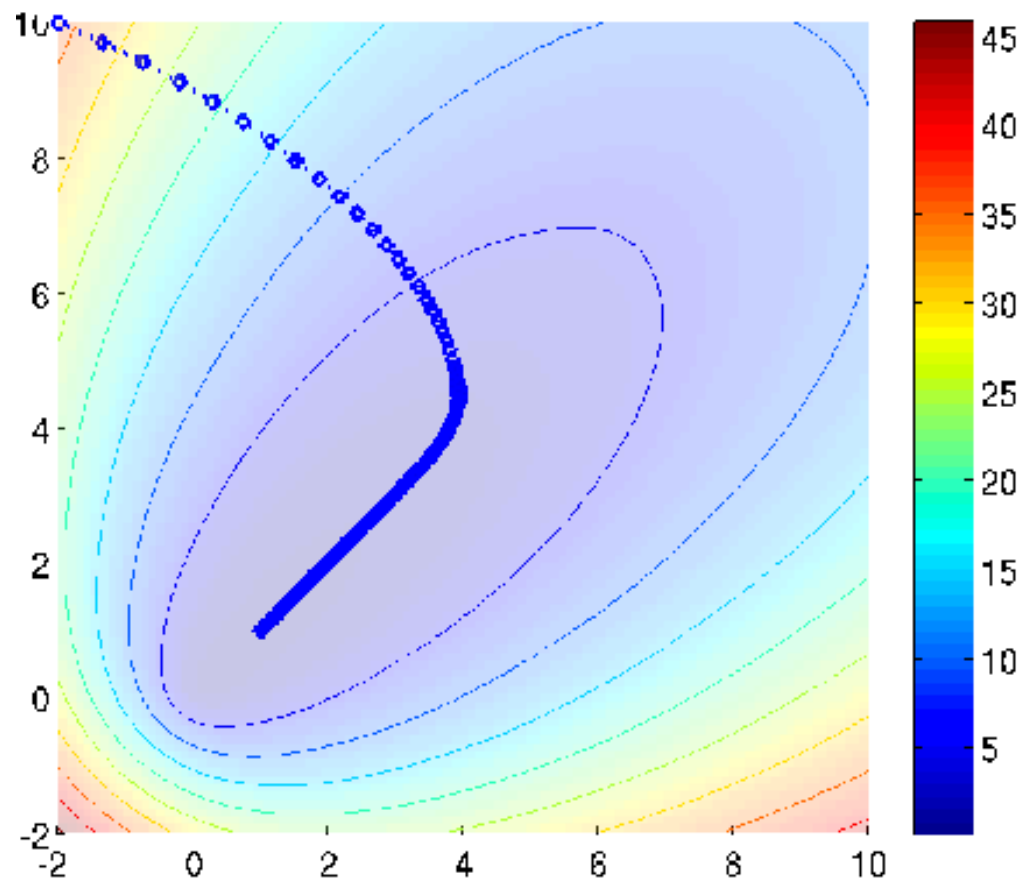
Gradient descent: stochastic

- Computes update for **each** example $x^{(i)}y^{(i)}$.
- Update equation: $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$

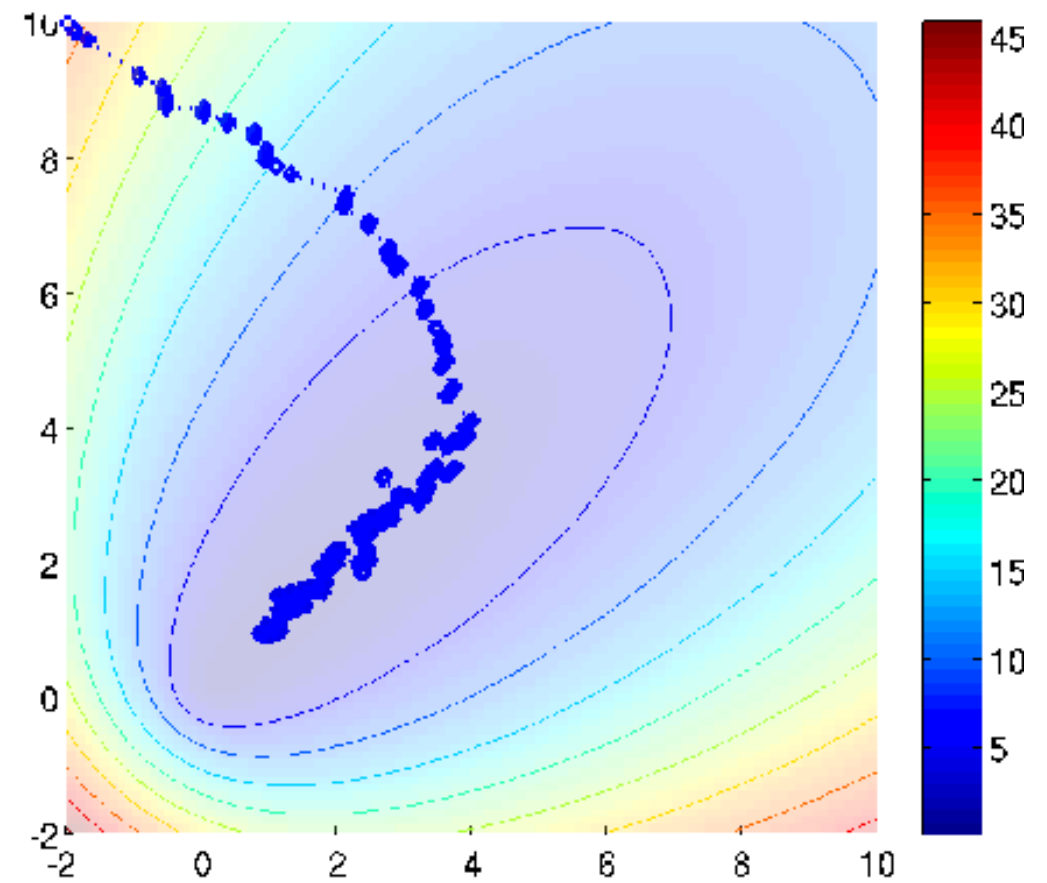
```
for i in range(nb_epochs):  
    np.random.shuffle(data)  
    for example in data:  
        params_grad = evaluate_gradient(  
            loss_function, example, params)  
        params = params - learning_rate * params_grad
```

- Pros
 - **Much faster** than batch gradient descent.
 - Allows **online learning**.
- Cons
 - **High variance** updates.

SGD vs. batch



(a) Gradient descent



(b) Stochastic gradient descent

Gradient descent: mini-batch

- Performs update for every **mini-batch** of n examples.
- Update equation: $\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)})$

```
for i in range(nb_epochs):  
    np.random.shuffle(data)  
    for batch in get_batches(data, batch_size=50):  
        params_grad = evaluate_gradient(  
            loss_function, batch, params)  
        params = params - learning_rate * params_grad
```

- Pros
 - **Reduces variance** of updates.
 - Can exploit **matrix multiplication** primitives.
- Cons
 - **Mini-batch size** is a hyperparameter. Common sizes are 50-256.
- Typically the algorithm of choice.
- Usually referred to as SGD even when mini-batches are used.

Batch Generator

```
def batch_generator(X, y, batch_size=32):  
    idx = np.arange(X.shape[0])  
    np.random.shuffle(idx)  
  
    n_batch = len(idx) // batch_size  
    for i in range(n_batch):  
        i_start = i * batch_size  
        i_stop = min((i + 1) * batch_size, len(idx))  
        yield X[i_start:i_stop, :], y[i_start:i_stop, :]
```

Gradient descent

Method	Accuracy	Update Speed	Memory Usage	Online Learning
Batch gradient descent	Good	Slow	High	No
Stochastic gradient descent	Good (with annealing)	High	Low	Yes
Mini-batch gradient descent	Good	Medium	Medium	Yes

Extended Gradient descent

- ① Momentum
- ② Nesterov accelerated gradient
- ③ Adagrad
- ④ Adadelata
- ⑤ RMSprop
- ⑥ Adam
- ⑦ Adam extensions

Momentum

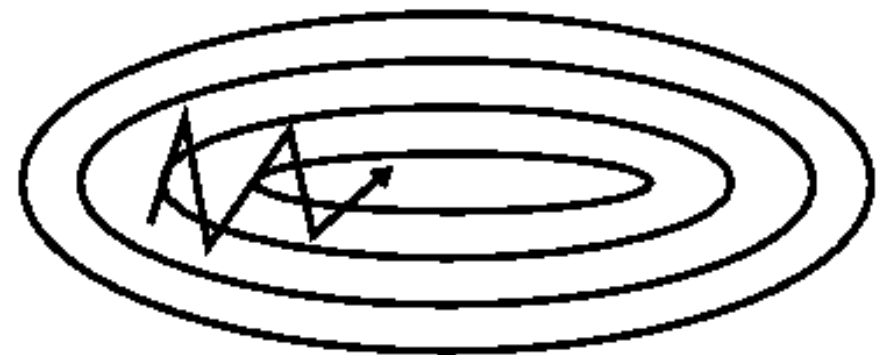
- SGD has trouble navigating **ravines**.
- Momentum [Qian, 1999] helps SGD **accelerate**.
- Adds a fraction γ of the update vector of the past step v_{t-1} to current update vector v_t . Momentum term γ is usually set to 0.9.

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$

$$\theta = \theta - v_t$$

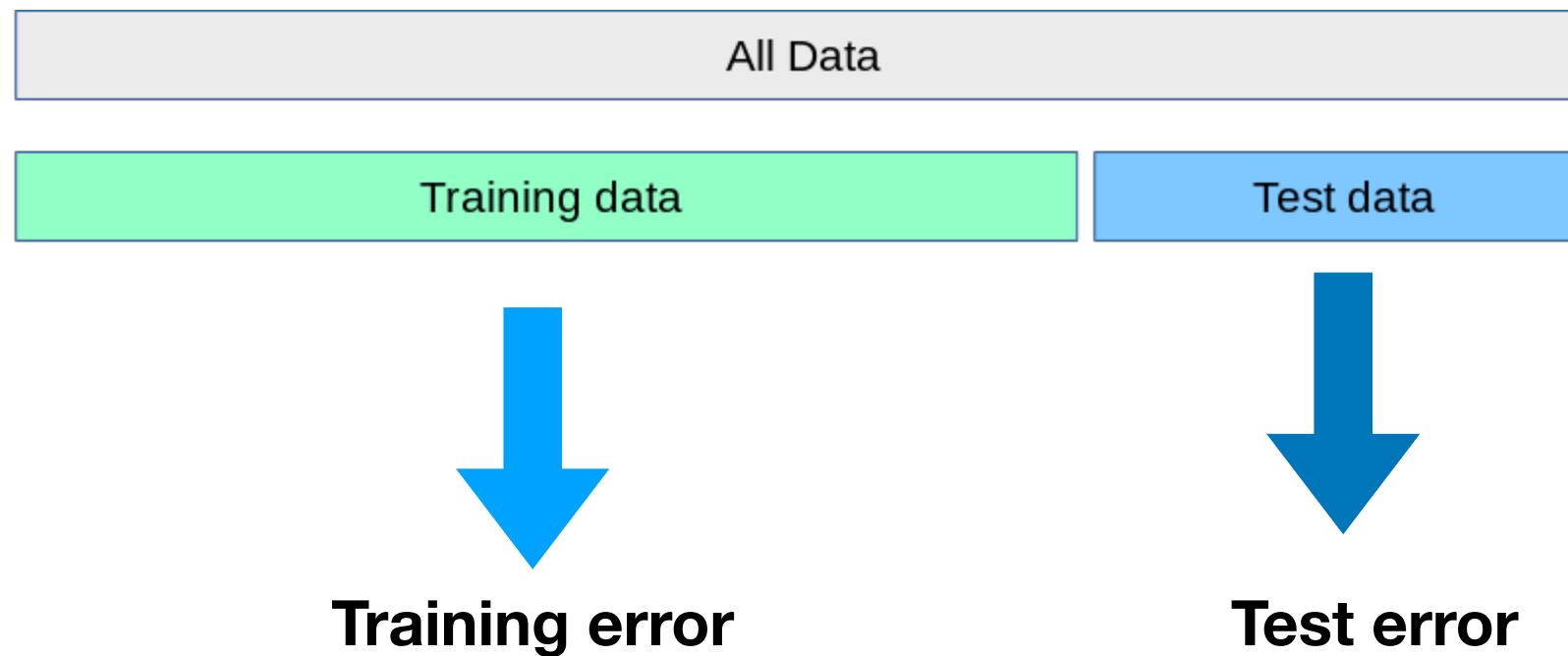


(a) SGD without momentum



(b) SGD with momentum

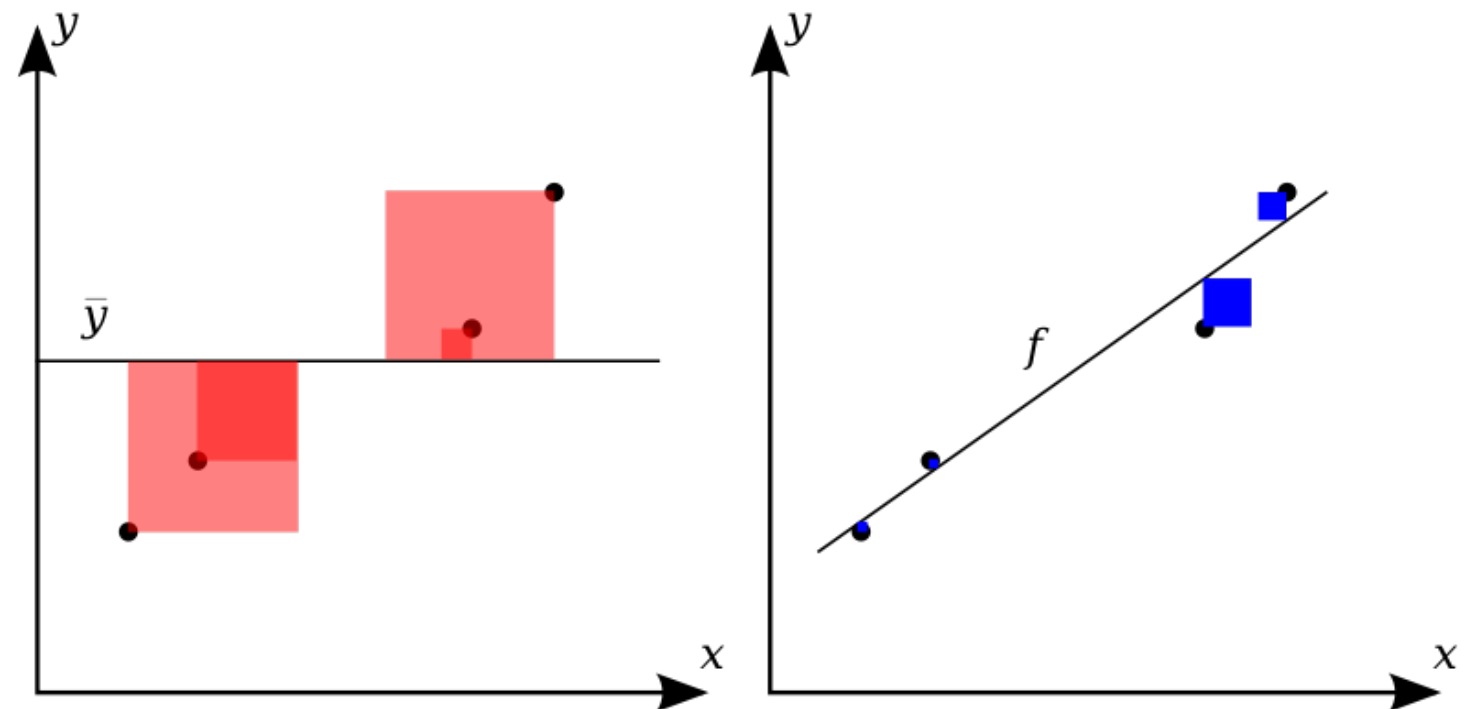
Phân chia Dữ liệu Huấn luyện và Dữ liệu Test



Đánh giá hiệu quả của mô hình hồi qui (Evaluation of Model)

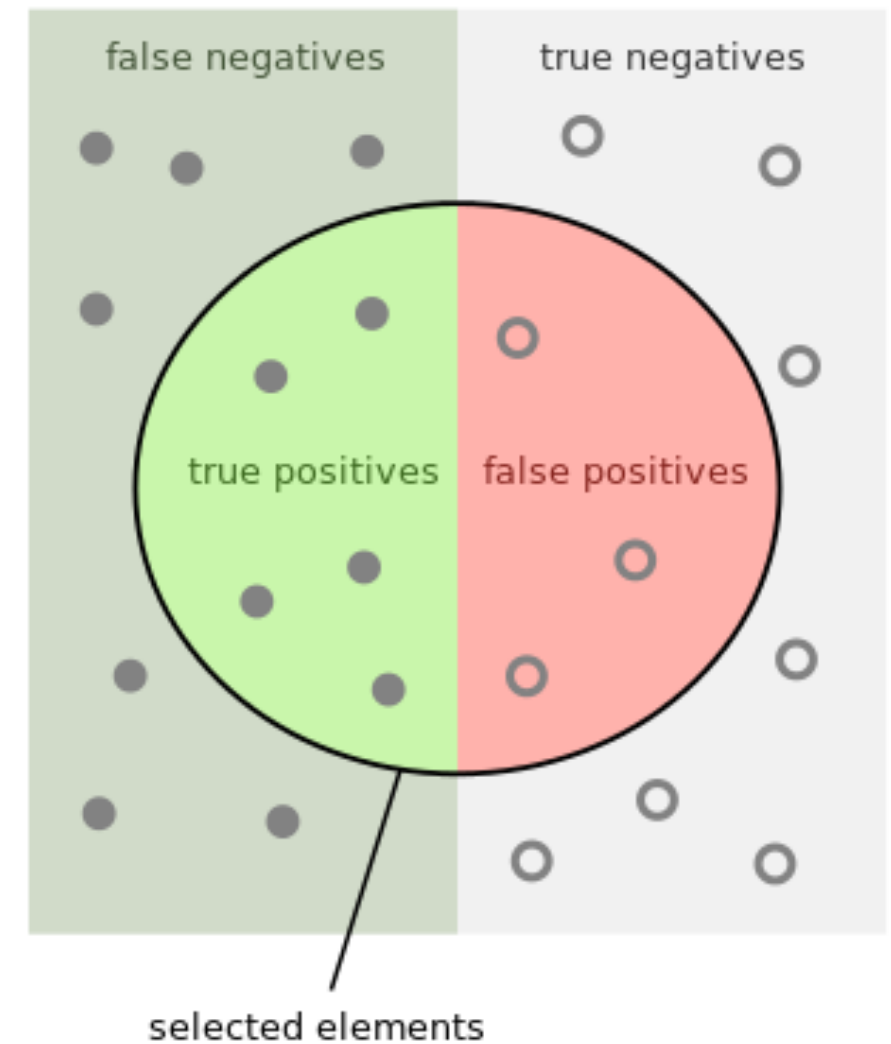
- Root Mean Square Error (RMSE)
- Mean Absolute Error (MAE)
- R² score (coefficient of determination)

$$R^2 = 1 - \frac{SSR}{SST}$$



Đánh giá hiệu quả của mô hình phân loại (Evaluation of Model)

- Accuracy
- Recall
- Precision



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$